UNISA

Universidade Santo Amaro

Universidade Santo Amaro – UNISA

Análise e Desenvolvimento de Sistemas

Técnicas de Programação

Análise Experimental de Algoritmos de Busca, Ordenação e Manipulação de Dados

Integrantes do grupo:

1. FENNIGUY TALLYS BARROS – RA: 5622743

ADS: Turma / 4º Semestre

Professora: Profa. Thamiris Neves

São Paulo – 2026

Resumo

Este trabalho investiga algoritmos de busca e ordenação e operações de inserção e remoção em estruturas de dados, utilizando bases sintéticas com 1.000, 10.000 e 100.000 registros. A base inicial foi obtida no Mockaroo e ampliada sinteticamente em Python. Foram configuradas 30 repetições por cenário executado, com cálculo de média, mediana e desvio padrão. Os experimentos foram executados em Windows 11 com Python 3.13.7 e PyCharm 2025.2.0.1. Na base desordenada de 100.000 registros, Quick Sort apresentou tempo médio de 268,947 ms entre os métodos de ordenação concluídos. O protocolo distinguiu cenários concluídos, interrompidos por limite de três segundos e não executados preventivamente. A análise indica que não existe um único algoritmo superior para todos os cenários.

Palavras-chave: algoritmos; Python; dados sintéticos; desempenho; Mockaroo; estruturas de dados.

Sumário

Resumo **1**

1 Introdução **4**

2 Objetivos **4**

2.1 Objetivo geral 4

2.2 Objetivos específicos 4

3 Metodologia **4**

3.1 Preparação das bases 4

3.2 Ambiente de execução 5

3.3 Protocolo de medição 5

3.4 Estatísticas utilizadas 5

4 Experimentos de busca **6**

4.1 Fundamentação 6

4.2 Resultados de 30 repetições 6

5 Experimentos de ordenação **8**

5.1 Algoritmos avaliados 8

5.2 Resultados de 30 repetições por cenário concluído 8

5.3 Interpretação 10

6 Experimentos de inserção e remoção **10**

6.1 Estruturas avaliadas 10

6.2 Inserção 10

6.3 Remoção 11

6.4 Interpretação estatística 12

7 Tempo total do laboratório **12**

8 Pergunta central do projeto **12**

8.1 Existe um algoritmo universalmente mais eficiente? 12

9 Discussão e limitações **13**

10 Conclusão **13**

11 Entregáveis e repositório **14**

Referências **15**

1 Introdução

A eficiência dos algoritmos depende do tamanho dos dados, de sua organização inicial e do padrão de operações. Este laboratório compara diferentes abordagens com medidas experimentais reproduzíveis, incluindo estatísticas descritivas dos tempos observados.

2 Objetivos

2.1 Objetivo geral

Comparar o desempenho de algoritmos de busca, ordenação e manipulação de dados em três escalas de entrada.

2.2 Objetivos específicos

- Avaliar buscas lineares e binárias, iterativas e recursivas.
- Comparar sete algoritmos de ordenação em bases ordenadas e desordenadas.
- Medir inserção e remoção em lista Python, lista encadeada e árvore binária de busca (BST).
- Calcular média, mediana e desvio padrão dos tempos em 30 repetições por cenário executado.
- Discutir escalabilidade, limitações, uso de memória e adequação a diferentes cargas de trabalho.

3 Metodologia

3.1 Preparação das bases

A base original com 1.000 registros foi produzida no Mockaroo. O programa `gerar_bases.py` criou bases sintéticas de 10.000 e 100.000 registros, a partir da amostra inicial, com versões ordenadas e desordenadas por `id_cliente`. Os campos são `id_cliente`, `nome`, `idade`, `cidade`, `estado`, `data_cadastro`, `numero_compras` e `valor_total`. As bases maiores não foram exportadas diretamente do Mockaroo. A amostra inicial contém valores do campo `estado` que não devem ser tratados automaticamente como unidades federativas validadas.

3.2 Ambiente de execução

Os testes foram realizados em um computador com as características apresentadas na Tabela 1. As informações foram registradas pelo programa experimental, com a versão do PyCharm confirmada no próprio ambiente de desenvolvimento.

Tabela 1: Configuração do ambiente de execução

Característica	**Especificação**
Sistema operacional	Windows 11 (build 26300)
Processador	Intel64 Family 6 Model 165, Stepping 3 (GenuineIntel)
Processadores lógicos	12
Memória RAM total	7,72 GB
Linguagem e versão	Python 3.13.7
Ambiente de desenvolvimento	PyCharm 2025.2.0.1
Repetições configuradas	30 por cenário executado
Limite de tempo da ordenação	3 segundos por execução
Tempo total do laboratório	285,93 segundos (aproximadamente 4 min e 46 s)

3.3 Protocolo de medição

Foram configuradas 30 repetições por cenário executado. Os tempos foram medidos com `time.perf_counter_ns()` e sintetizados por média aritmética, mediana e desvio padrão. As buscas incluem alvos primeiro, central, último e ausente; as buscas binárias são aplicadas apenas às bases ordenadas. Na ordenação, cada execução teve limite de três segundos. Cenários de custo potencialmente quadrático foram excluídos preventivamente em 100.000 registros. A construção inicial das estruturas não integra o tempo de inserção ou remoção individual. Os contadores de operações têm significados distintos entre estruturas. O tempo total inclui leitura, preparação, exportação e geração de gráficos, não apenas os algoritmos. A memória RAM total do equipamento foi registrada, mas o consumo de memória de cada algoritmo não foi medido separadamente.

3.4 Estatísticas utilizadas

Sejam $t_1, \ldots, t_k$ os tempos válidos, com $k = 30$ nos cenários concluídos. Calculou-se a média $\bar{t} = \frac{1}{k} \sum_{i=1}^{k} t_i$, a mediana (valor central após ordenação) e o desvio padrão conforme a implementação experimental. As três estatísticas permitem observar dispersão e influência de valores atípicos. Para casos não executados ou interrompidos, não foi atribuído tempo médio fictício.

4 Experimentos de busca

4.1 Fundamentação

A busca linear possui pior caso $O(n)$, enquanto a busca binária, em dados ordenados, possui pior caso $O(\log n)$. As versões recursivas podem apresentar sobrecusto do interpretador. Na base embaralhada, os nomes dos alvos referem-se aos IDs escolhidos, não necessariamente às posições físicas.

4.2 Resultados de 30 repetições

Tempos em microssegundos (μs). As colunas representam média, mediana e desvio padrão; todos os 72 cenários de busca possuem 30 repetições.

Base de 1.000 registros

Base	Algoritmo	Caso	Comp.	Média	Mediana	DP
desor	Linear	primeiro	800	66,85	71,40	17,85
desor	Linear	central	359	17,31	17,20	0,45
desor	Linear	ultimo	564	34,95	28,40	11,97
desor	Linear	ausente	1.000	61,99	54,65	17,90
desor	Linear rec.	primeiro	800	231,98	203,15	50,72
desor	Linear rec.	central	359	127,57	149,55	34,68
desor	Linear rec.	ultimo	564	166,78	147,50	36,67
desor	Linear rec.	ausente	1.000	282,90	258,60	47,61
orden	Linear	primeiro	1	0,48	0,40	0,39
orden	Linear	central	501	23,77	23,20	1,95
orden	Linear	ultimo	1.000	62,31	51,70	19,40
orden	Linear	ausente	1.000	57,62	51,90	15,21
orden	Linear rec.	primeiro	1	1,92	1,50	1,28
orden	Linear rec.	central	501	211,43	119,10	436,36
orden	Linear rec.	ultimo	1.000	250,77	244,00	18,48
orden	Linear rec.	ausente	1.000	275,84	246,10	61,47
orden	Binaria	primeiro	9	1,31	1,20	0,64
orden	Binaria	central	9	1,33	1,30	0,31
orden	Binaria	ultimo	10	1,39	1,40	0,18
orden	Binaria	ausente	10	1,46	1,40	0,15
orden	Binaria rec.	primeiro	9	1,98	1,70	0,78
orden	Binaria rec.	central	9	2,05	2,00	0,34
orden	Binaria rec.	ultimo	10	2,32	2,20	0,47
orden	Binaria rec.	ausente	10	2,40	2,30	0,49

Base de 10.000 registros

Base	Algoritmo	Caso	Comp.	Média	Mediana	DP
desor	Linear	primeiro	2.389	134,79	130,10	20,39
desor	Linear	central	2.431	176,74	146,05	51,81
desor	Linear	ultimo	6.834	394,49	383,20	48,39
desor	Linear	ausente	10.000	567,02	561,10	15,00
desor	Linear rec.	primeiro	2.389	603,80	587,65	39,98
desor	Linear rec.	central	2.431	624,47	609,70	33,86
desor	Linear rec.	ultimo	6.834	1.744,59	1.716,40	67,47
desor	Linear rec.	ausente	10.000	2.843,33	2.711,15	259,96
orden	Linear	primeiro	1	0,37	0,30	0,28
orden	Linear	central	5.001	283,64	282,90	5,10
orden	Linear	ultimo	10.000	584,52	571,50	43,59
orden	Linear	ausente	10.000	583,72	580,25	20,20

Base	Algoritmo	Caso	Comp.	Média	Mediana	DP
orden	Linear rec.	primeiro	1	2,39	2,00	1,21
orden	Linear rec.	central	5.001	1.416,33	1.394,00	140,44
orden	Linear rec.	ultimo	10.000	2.868,07	2.766,50	255,79
orden	Linear rec.	ausente	10.000	2.996,84	2.942,20	222,14
orden	Binaria	primeiro	13	3,22	3,00	1,01
orden	Binaria	central	13	2,80	3,05	1,01
orden	Binaria	ultimo	14	2,28	1,70	2,93
orden	Binaria	ausente	14	2,38	1,85	2,73
orden	Binaria rec.	primeiro	13	2,57	2,50	0,38
orden	Binaria rec.	central	13	3,78	2,70	3,83
orden	Binaria rec.	ultimo	14	3,45	2,80	2,97
orden	Binaria rec.	ausente	14	5,79	5,60	0,60

Base de 100.000 registros

Base	Algoritmo	Caso	Comp.	Média	Mediana	DP
desor	Linear	primeiro	95.781	6.354,53	6.383,00	743,59
desor	Linear	central	65.729	4.264,73	4.145,50	372,69
desor	Linear	ultimo	81.504	4.903,78	4.763,90	415,68
desor	Linear	ausente	100.000	5.875,85	5.706,75	638,03
desor	Linear rec.	primeiro	95.781	25.128,01	24.845,70	2.400,09
desor	Linear rec.	central	65.729	17.402,97	17.207,00	712,69
desor	Linear rec.	ultimo	81.504	20.439,55	19.998,00	829,33
desor	Linear rec.	ausente	100.000	25.688,26	24.958,50	1.819,09
orden	Linear	primeiro	1	0,38	0,30	0,33
orden	Linear	central	50.001	2.929,25	2.902,45	154,22
orden	Linear	ultimo	100.000	5.938,35	5.884,85	258,37
orden	Linear	ausente	100.000	8.625,92	8.406,10	2.559,99
orden	Linear rec.	primeiro	1	2,57	2,40	0,57
orden	Linear rec.	central	50.001	14.329,95	14.026,85	1.619,29
orden	Linear rec.	ultimo	100.000	26.571,14	25.860,15	1.719,01
orden	Linear rec.	ausente	100.000	25.601,96	24.985,90	1.693,81
orden	Binaria	primeiro	16	2,12	1,90	0,91
orden	Binaria	central	16	2,18	2,10	0,59
orden	Binaria	ultimo	17	2,21	2,10	0,38
orden	Binaria	ausente	17	2,40	2,20	0,46
orden	Binaria rec.	primeiro	16	3,08	3,00	0,39
orden	Binaria rec.	central	16	3,30	3,20	0,35
orden	Binaria rec.	ultimo	17	3,44	3,40	0,27
orden	Binaria rec.	ausente	17	3,55	3,50	0,29

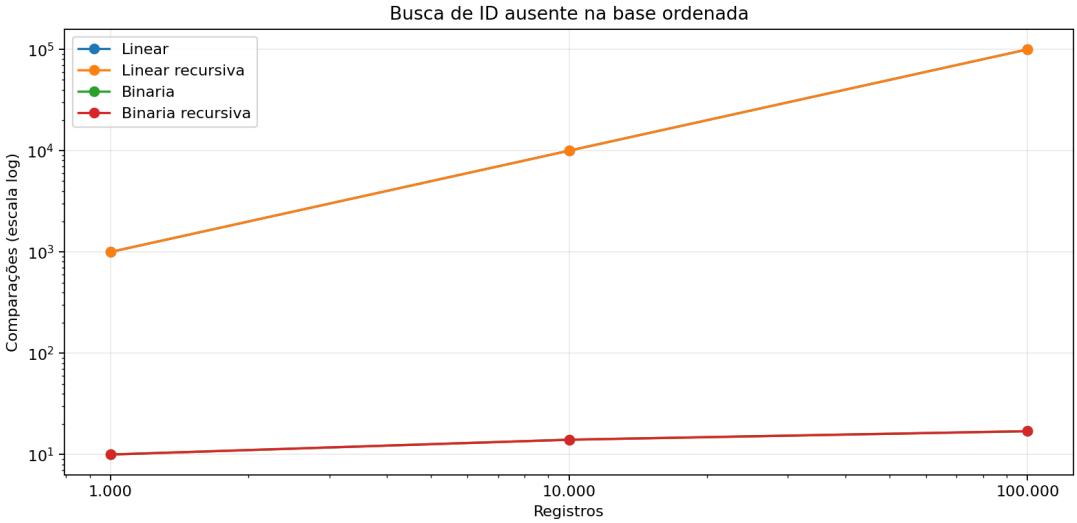


Figura 1: Número de comparações na busca de ID ausente em base ordenada.

Na base ordenada com 100.000 registros, a busca linear por um ID ausente realizou 100.000 comparações, enquanto a busca binária realizou 17. O resultado ilustra a diferença entre crescimento linear e logarítmico.

5 Experimentos de ordenação

5.1 Algoritmos avaliados

Foram avaliados Bubble, Selection, Insertion, Shell, Merge, Quick e Heap Sort. O Bubble otimizado e o Insertion podem se beneficiar de entradas já ordenadas. O Quick Sort implementado com pivô final pode apresentar pior caso quadrático em dados ordenados. O Shell Sort depende da sequência de incrementos.

5.2 Resultados de 30 repetições por cenário concluído

Tempos em milissegundos (ms). "Limite 3 s" indica tempo excedido; "Não executado" indica exclusão preventiva. O símbolo "–" não equivale a zero. Comparações e movimentações correspondem aos contadores definidos na implementação.

Base ordenada

N	Algoritmo	Status	Média	Mediana	DP	Comp.	Mov.
1.000	Bubble	OK	0,094	0,085	0,024	999	0
1.000	Selection	OK	31,291	30,600	1,876	499.500	0
1.000	Insertion	OK	0,207	0,170	0,088	999	999
1.000	Shell	OK	1,081	1,043	0,096	8.006	8.006
1.000	Merge	OK	1,800	1,757	0,126	5.052	10.000
1.000	Quick	OK	38,946	38,544	1,544	499.500	0
1.000	Heap	OK	2,654	2,538	0,435	17.583	19.416
10.000	Bubble	OK	0,963	0,924	0,135	9.999	0
10.000	Selection	Limite 3 s	–	–	–	–	–
10.000	Insertion	OK	1,788	1,791	0,076	9.999	9.999
10.000	Shell	OK	17,470	17,201	1,335	120.005	120.005
10.000	Merge	OK	25,244	24,683	1,805	71.712	140.000
10.000	Quick	Limite 3 s	–	–	–	–	–
10.000	Heap	OK	35,070	34,334	2,565	244.460	263.912
100.000	Bubble	Não executado	–	–	–	–	–
100.000	Selection	Não executado	–	–	–	–	–
100.000	Insertion	Não executado	–	–	–	–	–
100.000	Shell	OK	225,690	228,484	11,904	1.500.006	1.500.006
100.000	Merge	OK	304,241	302,976	11,025	877.968	1.700.000
100.000	Quick	Não executado	–	–	–	–	–
100.000	Heap	OK	450,929	448,925	12,688	3.112.517	3.301.708

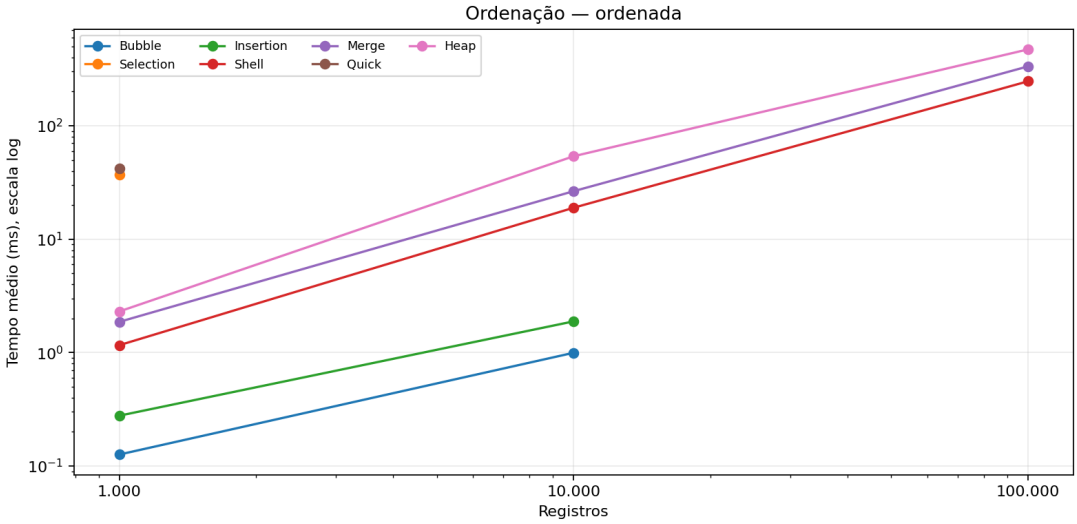


Figura 2: Tempos médios dos algoritmos concluídos na base ordenada.

Base desordenada

N	Algoritmo	Status	Média	Mediana	DP	Comp.	Mov.
1.000	Bubble	OK	66,440	65,484	5,752	499.035	481.040
1.000	Selection	OK	31,329	31,013	1,554	499.500	1.980
1.000	Insertion	OK	32,674	32,359	1,745	241.511	241.519
1.000	Shell	OK	2,456	2,297	0,497	14.968	15.466
1.000	Merge	OK	2,292	2,249	0,190	8.729	10.000
1.000	Quick	OK	1,474	1,429	0,150	10.604	9.484
1.000	Heap	OK	2,334	2,303	0,113	16.811	18.140
10.000	Bubble	Limite 3 s	–	–	–	–	–
10.000	Selection	Limite 3 s	–	–	–	–	–
10.000	Insertion	Limite 3 s	–	–	–	–	–
10.000	Shell	OK	43,404	42,489	2,640	263.954	269.042
10.000	Merge	OK	33,674	31,382	6,305	123.626	140.000
10.000	Quick	OK	21,149	19,938	4,037	152.069	138.510
10.000	Heap	OK	35,768	34,477	3,371	235.096	248.058
100.000	Bubble	Não executado	–	–	–	–	–
100.000	Selection	Não executado	–	–	–	–	–
100.000	Insertion	Não executado	–	–	–	–	–
100.000	Shell	OK	805,127	764,562	118,567	4.358.927	4.409.441
100.000	Merge	OK	411,662	403,126	29,580	1.566.472	1.700.000
100.000	Quick	OK	268,947	267,167	7,629	1.947.436	1.936.330
100.000	Heap	OK	482,039	474,539	27,023	3.019.514	3.149.706

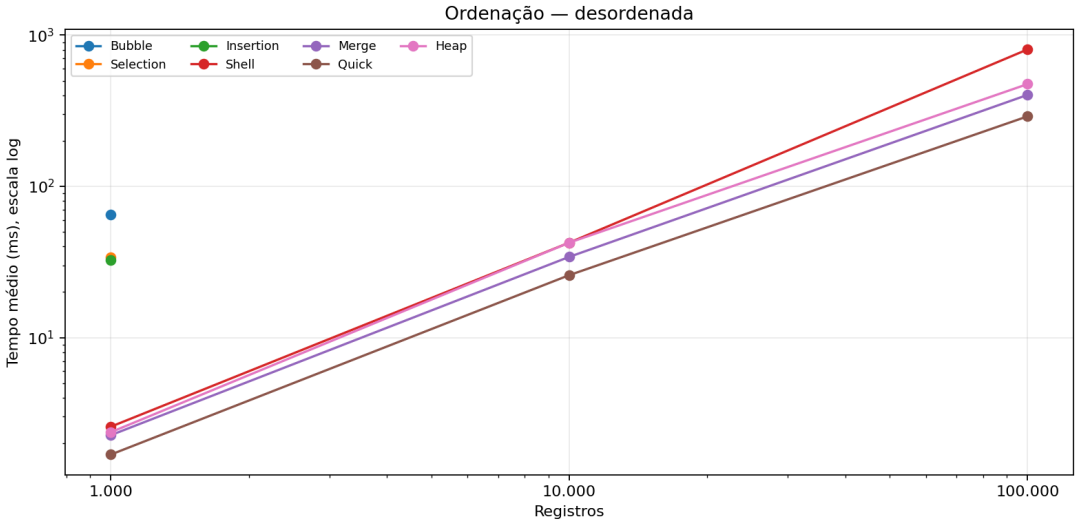


Figura 3: Tempos médios dos algoritmos concluídos na base desordenada.

5.3 Interpretação

Na base desordenada de 100.000 registros, o Quick Sort apresentou o menor tempo médio entre os algoritmos executados (268,947 ms), seguido por Merge, Heap e Shell. Na base ordenada de 100.000 registros, o Shell Sort foi o mais rápido entre os executados (225,690 ms). Essas conclusões referem-se às implementações e à máquina utilizadas. Cenários sem medições válidas não são considerados comparações de velocidade.

6 Experimentos de inserção e remoção

6.1 Estruturas avaliadas

Foram usadas lista Python, lista simplesmente encadeada e árvore binária de busca (BST) sem balanceamento automático. Na lista Python, o contador representa deslocamentos; na lista encadeada, visitas a nós; na BST, comparações. Esses contadores não são diretamente equivalentes. Tempos em microssegundos (μs).

6.2 Inserção

N	Estrutura	Pos.	Média	Mediana	DP	Operações
1.000	Lista Py	Inicio	7,73	0,60	38,54	1.000
1.000	Lista Py	Meio	0,52	0,50	0,23	500
1.000	Lista Py	Fim	0,35	0,30	0,13	0
1.000	Encadeada	Inicio	1,38	0,60	3,49	0
1.000	Encadeada	Meio	16,52	16,30	0,75	500
1.000	Encadeada	Fim	35,72	35,60	0,23	1.000
1.000	BST	Por ID	1,45	1,20	0,77	5
10.000	Lista Py	Inicio	5,21	3,90	2,72	10.000
10.000	Lista Py	Meio	2,17	2,10	0,32	5.000

N	Estrutura	Pos.	Média	Mediana	DP	Operações
10.000	Lista Py	Fim	0,57	0,50	0,19	0
10.000	Encadeada	Inicio	1,56	1,25	0,75	0
10.000	Encadeada	Meio	219,31	198,90	54,25	5.000
10.000	Encadeada	Fim	425,12	404,00	46,96	10.000
10.000	BST	Por ID	4,05	3,60	1,55	8
100.000	Lista Py	Inicio	194,16	204,30	74,95	100.000
100.000	Lista Py	Meio	162,72	152,85	95,66	50.000
100.000	Lista Py	Fim	131,38	135,00	77,46	0
100.000	Encadeada	Inicio	5,29	4,60	3,58	0
100.000	Encadeada	Meio	2.288,29	2.198,40	214,68	50.000
100.000	Encadeada	Fim	4.670,05	4.434,10	575,32	100.000
100.000	BST	Por ID	8,04	8,05	1,62	14

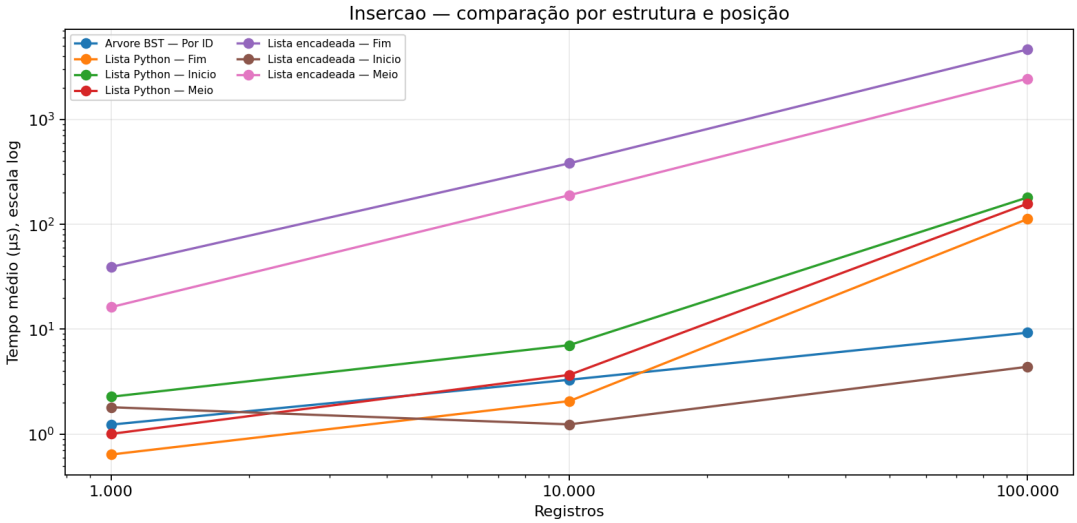


Figura 4: Tempos medianos de inserção por estrutura e tamanho de base.

6.3 Remoção

N	Estrutura	Pos.	Média	Mediana	DP	Operações
1.000	Lista Py	Inicio	6,22	0,40	31,56	999
1.000	Lista Py	Meio	6,23	0,40	32,06	499
1.000	Lista Py	Fim	0,23	0,20	0,08	0
1.000	Encadeada	Inicio	0,53	0,40	0,58	0
1.000	Encadeada	Meio	16,46	16,30	0,55	500
1.000	Encadeada	Fim	36,51	36,40	0,42	999
1.000	BST	Por ID	3,02	1,55	6,49	12
10.000	Lista Py	Inicio	2,00	2,00	0,12	9.999
10.000	Lista Py	Meio	1,69	1,75	0,59	4.999
10.000	Lista Py	Fim	0,49	0,40	0,23	0
10.000	Encadeada	Inicio	0,88	0,70	0,47	0
10.000	Encadeada	Meio	234,26	205,70	60,05	5.000
10.000	Encadeada	Fim	450,14	437,95	39,79	9.999
10.000	BST	Por ID	3,97	3,45	2,19	17
100.000	Lista Py	Inicio	21,95	18,70	10,07	99.999
100.000	Lista Py	Meio	9,63	9,20	1,76	49.999
100.000	Lista Py	Fim	0,72	0,50	0,51	0
100.000	Encadeada	Inicio	3,86	3,55	1,51	0
100.000	Encadeada	Meio	2.249,42	2.233,15	112,07	50.000
100.000	Encadeada	Fim	4.694,21	4.484,75	1.162,62	99.999
100.000	BST	Por ID	9,55	8,85	2,64	26

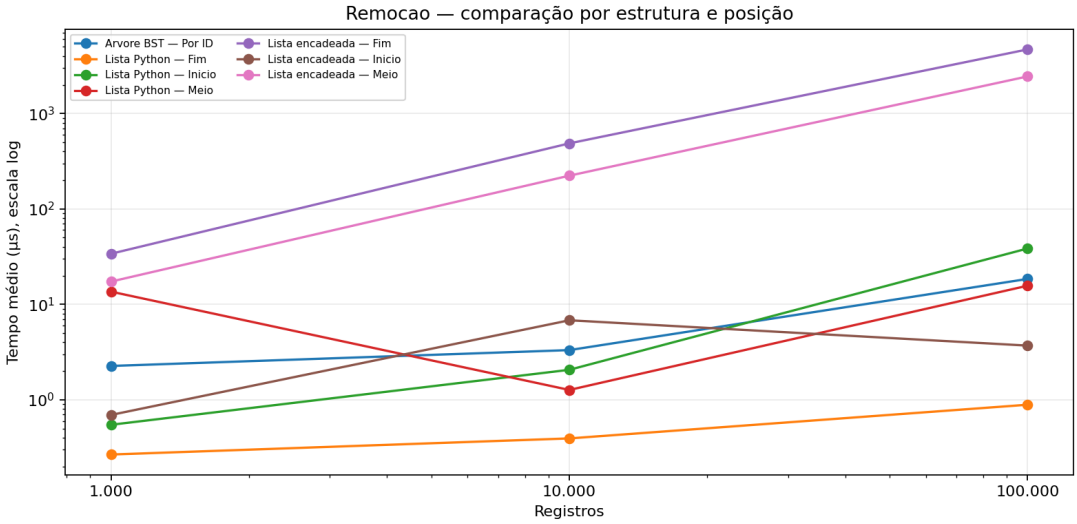


Figura 5: Tempos medianos de remoção por estrutura e tamanho de base.

6.4 Interpretação estatística

Média e mediana podem divergir em operações muito rápidas devido a variações pontuais de execução. Por exemplo, na inserção no início da lista Python com 1.000 registros, a média foi 7,73 μs, a mediana 0,60 μs e o desvio padrão 38,54 μs. Isso sugere a influência de medições atípicas. O tempo de construção das estruturas não foi incluído nas operações isoladas.

7 Tempo total do laboratório

A execução integral do laboratório consumiu 285,93 segundos (aproximadamente 4 minutos e 46 segundos). Esse tempo inclui a leitura das bases, as medições, o tratamento de casos com limite de tempo, a exportação dos CSVs e a criação dos gráficos; não deve ser interpretado como o tempo de um algoritmo isolado.

8 Pergunta central do projeto

8.1 Existe um algoritmo universalmente mais eficiente?

Não. A escolha depende do volume de registros, da organização inicial, da frequência de consultas, inserções e remoções, do custo de preparar os dados e das restrições de tempo e memória.

- **Consultas frequentes:** em bases já ordenadas, a busca binária reduz comparações; entretanto, ordenar previamente tem custo e pode não compensar poucas consultas.

- **Ordenação:** a eficiência varia com o estado inicial. Bubble otimizado e Insertion tiveram bons resultados em bases ordenadas; Quick foi competitivo em bases desordenadas, mas a estratégia de pivô final é vulnerável ao pior caso.
- **Inserções e remoções:** operações no início, meio e fim têm custos distintos em listas Python e encadeadas. A BST permite busca por chave, mas depende de sua forma e não é balanceada.
- **Memória:** diferentes algoritmos e estruturas têm custos espaciais teóricos distintos; este laboratório não mediu o consumo de memória de cada algoritmo. A memória RAM do equipamento foi registrada, mas não representa uso de memória por operação.
- **Tempo total do sistema:** 285,93 segundos inclui etapas externas aos algoritmos, de modo que não é uma métrica comparável diretamente aos tempos de ordenação, busca ou manipulação.

9 Discussão e limitações

A ampliação das bases evidenciou diferenças de escalabilidade. A utilização de 30 repetições permitiu registrar média, mediana e dispersão, mas não elimina ruídos do sistema operacional. As bases maiores foram geradas a partir de uma amostra sintética inicial; a BST não é balanceada; os contadores de operações diferem entre estruturas; e os casos interrompidos ou excluídos não oferecem tempo exato. Não houve instrumentação de consumo de memória por algoritmo nem simulação de cargas reais com frequências variadas de consultas, inserções e remoções. Tais aspectos são recomendações para estudos futuros.

10 Conclusão

O laboratório mostrou que não há algoritmo universalmente superior. A busca binária foi vantajosa em número de comparações para consultas em bases ordenadas; métodos de ordenação apresentaram respostas diferentes conforme o estado dos dados; e inserção e remoção variaram com a estrutura e a posição. A execução com 30 repetições por cenário concluído produziu medidas de tendência central e dispersão, tornando a análise mais informativa. Os cenários limitados ou não executados foram identificados de forma transparente. O projeto reúne código, bases, resultados, gráficos e documentação, com repositório público para consulta.

11 Entregáveis e repositório

O projeto disponibiliza código-fonte Python, bases CSV, resultados CSV, gráficos, arquivo README e relatório PDF. Repositório: https://github.com/FenniguyTallys/Laboratorio-Algoritmos-UNISA. Para reproduzir a análise, executar `gerar_bases.py` e `laboratorio_30_repeticoes.py`, com as dependências instaladas, respeitando a estrutura de pastas indicada no README. A publicação dos novos resultados no repositório deve ser feita após sua conferência.

Referências

CORMEN, Thomas H. et al. **Introduction to Algorithms**. 4. ed. Cambridge: MIT Press, 2022.

MOCKAROO. **Random Data Generator and API Mocking Tool**. Disponível em: https://www.mockaroo.com/. Acesso em: 6 nov. 2026.